Quantitative Trading with Python: A Top-Down Approach

Jack Forden

March 2026

Contents

1 Physics Before Formulas **1**

1.1 What Is the Market? 1

1.1.1 The Market's Structure 2

1.1.2 The Digital Revolution and the NASDAQ Era 5

1.1.3 The Market as a Real-Time Decision Field 5

1.1.4 Primary vs Secondary Markets 6

1.1.5 Core Market Participants 6

1.1.6 Dealers and Market Making 6

1.1.7 Brokers and Order Routing 6

1.1.8 Financial Intermediaries 7

1.2 System Lens 7

2 From Intuition to Measurement **8**

2.1 What We Measure 8

3 Machine Learning **9**

3.1 Principles of Machine Learning 10

3.1.1 What Makes a Machine Learn 12

3.2 Supervised Learning 13

3.3 Decision Trees 17

3.3.1 Overfitting 19

3.3.2 Pruning 20

3.3.3 Choosing the Best Attribute 21

3.4 Building the Financial ML Pipeline 24

3.4.1 Technical Indicators 27
3.4.2 From Raw Data to Features 28
3.4.3 Finding Labels: Fixed-Horizon Method 29
3.4.4 Finding Labels: The Triple-Barrier Method 30
3.4.5 Meta Labeling 30
3.4.6 Finalizing the Feature Matrix 32
3.5 Evaluating the Model 34
3.5.1 Train/Test Split 34
3.5.2 The Confusion Matrix 35
3.5.3 Precision, Recall, and the Cost of Being Wrong 37

Chapter 1

Physics Before Formulas

Today's global financial market is arguably one of the largest engineered systems in existence: millions of instruments, fragmented venues, market makers, brokers, clearing systems, data centers, scripts, and autonomous agents interacting in nanoseconds. The New York Stock Exchange, which started when 24 brokers signed the Buttonwood Agreement in 1792, has grown into a global network of exchanges and trading venues that operate around the clock. In the modern era, the conception of a "market" has expanded to include not only traditional stock exchanges but also electronic trading platforms, dark pools, decentralized finance (DeFi) networks, prediction markets.

When viewing the market as an independent patchwork of venues, participants, and instruments, it is easy to get lost in the details. The goal of this chapter is the opposite: to abstract the market into a system that can be reasoned about, and then use that abstraction to guide the design of trading strategies that are robust by construction, not by accident.

This chapter provides a broad overview to see the forest through the trees: terminology first, then market edge (participants and workflows), market core (venues, execution, and data flow), and finally the intelligence layer (models, agents, risk controls, and failure modes).

1.1 What Is the Market?

When most people hear "the market," they imagine a building, a bell, or a chart on a screen. That picture is not wrong, but it is incomplete enough to hurt you if you trade from it. The market is a network of interacting agents and venues that collectively determine prices and allocate capital, each participant operating under their own objectives, constraints, and information.

To start, we will define the market from a high level perspective, focusing on its structure, participants, and the flow of information and capital. The second half of the chapter will focus on the market from a system engineering perspective, where we explore the market as a complex adaptive system that can be modeled, analyzed, and optimized.

1.1.1 The Market's Structure

The New York Stock Exchange traces back to the Buttonwood Agreement, signed by 24 brokers in New York on May 17, 1792. In modern terms, Buttonwood functioned like a protocol: it defined who could trade with whom and standardized commissions. The point was trust and control, not speed. By requiring a broker for every trade, the agreement ensured that every transaction passed through a verified member who was personally responsible for the deal's success.

Prior to the Buttonwood Agreement, trading was far more informal, with brokers negotiating deals directly with each other in coffee houses and other ad hoc venues. The lack of structure often led to disputes, inconsistent pricing, and a fear of **counterparty risk**: there was no guarantee that the person on the other side of a trade would actually deliver the stock or the cash.

By formalizing the rules, the Buttonwood signers helped establish trust and confidence in the financial system. In many ways, the agreement functioned as an early whitelist, where brokers served as transparent validator nodes in a distributed network. That newfound confidence proved crucial for growth: it enabled more efficient price discovery, increased liquidity, and attracted broader participation from investors. The market expanded in lockstep with the American economy as more companies sought to raise capital and more investors looked for opportunity.

In the early days, investors did not directly communicate with the exchange. Instead, they relied on brokerage offices or written instructions to place orders. On the exchange floor, brokers gathered around designated trading posts shouting bids and offers. Only when two brokers agreed on a price was the trade recorded in notebooks.

Information about prices and trades then had to be transmitted back to investors through messengers, couriers, or telegraph updates. This process introduced unavoidable delays between activity on the exchange floor and the information available to distant investors.

In effect, the market had encountered its first **latency arbitrage** problem:

participants with faster access to information could exploit those with slower access. An investor located in Boston, for example, might have to wait hours or even days for a courier to deliver trade information from New York, while a trader on the floor could act on that information immediately. The manual communication infrastructure that supported early trading had been designed for a much smaller market, and it struggled to keep pace with the accelerating volume of transactions. This latency gap meant an investor might attempt to buy a stock at a certain price, but by the time their order reached the exchange, the price could already have changed, resulting in a missed opportunity or a less favorable trade.

To address this gap, Edward A. Calahan invented the ticker tape machine, which transmitted stock prices over existing telegraph infrastructure. The device was one of the earliest examples of *near-real-time data dissemination*. Ticker tape machines functioned as specialized telegraphs whose sole purpose was to print stock symbols, prices, and volume onto a narrow paper strip. By 1929, ticker tape machines were installed in brokerage offices, investor's homes, and financial institutions across the United States, providing a continuous stream of market data to investors and traders. The result was a dramatic improvement in the speed and accuracy of information flow from Wall Street to the rest of the country. For the first time, investors could make decisions based on near-real-time data, rather than relying on delayed reports. In many ways, the ticker tape transformed financial markets from a localized trading floor into a distributed information network. Ticker tape was not without its physical limitations. Machines could only print a limited number of characters per second, and the paper strips were prone to jamming and tearing. Under normal conditions these issues were manageable, and by 1929 improved models could relay data at 285 characters per minute. But the system had a hard ceiling: when trading volume outpaced print speed, the tape fell behind, and every investor reading it was making decisions on stale prices.

From a systems perspective, the stock market had become an information network whose demand for data transmission was beginning to exceed its available bandwidth. October 1929 proved the point catastrophically. As trading volume surged past what the ticker infrastructure could handle, reporting delays grew from minutes to hours, and a feedback loop of panic selling and stale data amplified the crash far beyond what the underlying economic conditions alone would have caused.

CASE HISTORY The 1929 Crash as an Information-System Failure

Leading up to October 1929, several macroeconomic and political factors were already placing pressure on the financial system. In August 1929, the Federal Reserve raised interest rates from 5% to 6%, tightening credit conditions across the economy. Financial forecaster Roger Babson had publicly warned of a coming collapse, an event that later became known as the "Babson Break." At the same time, international instability, including the Hatry Crisis in the United Kingdom, caused British investors to withdraw capital from American markets. Domestically, income inequality meant that a relatively small portion of the population controlled most investment capital, limiting the pool of new buyers needed to sustain rising prices. Debates surrounding the proposed Smoot–Hawley Tariff Act also created fears of a global trade conflict.

These factors increased market volatility and trading activity, pushing the ticker tape system toward its operational limits.

During the week of the crash, trading volume surged to unprecedented levels. On October 21, 1929, approximately six million shares were traded, overwhelming ticker systems and causing reporting delays of nearly 100 minutes. By October 24, later known as *Black Thursday*, trading volume reached 12.9 million shares, and ticker delays expanded to nearly four hours.

For investors relying on ticker tape updates, the information they were receiving was no longer current market data, but rather a delayed record of trades that had occurred hours earlier. Investors observing falling prices on the ticker believed the market was collapsing in real time, even though the prices they were seeing reflected conditions from much earlier in the trading session. As panic selling accelerated, the volume of trades increased even further, widening the gap between actual market activity and the data being disseminated to the public.

By October 28 (*Black Monday*), trading volume reached 9.2 million shares with ticker delays approaching three hours. The following day, October 29, 1929, would become known as *Black Tuesday*: 16.4 million shares were traded, an unprecedented level of activity for the era.

The infrastructure responsible for distributing price data could no longer keep pace with the rate at which new trades were occurring. What began as an economic correction quickly escalated into a systemic information failure, where millions of investors were making decisions based on data that was already hours out of date.

Viewed through a modern lens, the events of October 1929 illustrate a critical principle: price discovery is fundamentally constrained by the speed at which information can propagate through the system. The crash did not just expose a limitation of ticker tape; it revealed that any market whose data infrastructure cannot keep pace with its trading activity is structurally fragile. Solving this problem would require more than faster printers. It would require a fundamentally different approach to how prices were generated, matched, and distributed.

1.1.2 The Digital Revolution and the NASDAQ Era

The answer, when it finally arrived, was electronic execution. NASDAQ launched in 1971 as the first fully electronic stock market, replacing the physical ticker with digital price quotation and, eventually, automated order matching.

Where ticker tape had been a one-way broadcast—prices printed onto paper and shipped outward—electronic systems closed the loop. Quotes, orders, and confirmations could travel the same network in both directions, collapsing the gap between observing a price and acting on it. Over time, electronic routing, matching, and reporting became normal across venues. The same information bottleneck that had amplified the 1929 crash was, in principle, eliminated: digital systems could scale throughput in ways that mechanical printers never could.

Digital systems also made market data more machine-readable and more available at scale, which is exactly what quantitative research needed. But computerization did not magically "clean" the data. It introduced new forms of noise, latency effects, and cross-venue inconsistency that quants still have to model explicitly.

1.1.3 The Market as a Real-Time Decision Field

In this course, we treat the market as a high-dimensional, real-time decision environment.

Every tick is a decision event. It may be generated by a human, an automated strategy, or a hybrid workflow. Either way, each print is an action taken under constraints, not a timeless truth.

1.1.4 Primary vs Secondary Markets

Primary markets are where new securities are issued (for example, IPO allocation and initial sale). Secondary markets are where existing claims are continuously repriced and transferred.

Most daily equity trading you see is secondary-market activity. That flow is less about raising new capital for firms and more about ongoing price discovery among holders with competing views.

1.1.5 Core Market Participants

The high-level player map from lecture is the right starting point:

- Dealers/market makers: provide liquidity, quote two-sided prices, and manage inventory risk.
- Brokers: route and execute customer orders, with explicit obligations around execution quality.
- Speculators (including retail, quants, and students): seek returns from directional, statistical, or structural edges.

If you only look at exchanges and ignore these actors, you are not modeling the market. You are modeling a dashboard.

1.1.6 Dealers and Market Making

Dealers do not behave like long-only investors. Their core business is facilitating flow while managing inventory and spread risk.

In practice, many realized fills are the product of a dealer's willingness to hold risk briefly in exchange for spread economics and flow scale.

1.1.7 Brokers and Order Routing

Brokers are the transport layer between client intent and executable flow. Route choice, venue selection, and timing directly affect realized outcomes.

"Best execution" is not a slogan. It is an operational constraint with measurable consequences for slippage, fill quality, and strategy PnL.

1.1.8 Financial Intermediaries

Beyond brokers and dealers, large intermediaries (banks, funds, and asset managers) pool capital, transform exposures, and repackage risk.

These institutions change both the shape of order flow and the risk topology of the system. Ignoring them leads to naive causal stories.

1.2 System Lens

Anchor each implementation step to market mechanics first.

Chapter 2

From Intuition to Measurement

This placeholder chapter is ready for the next draft.

2.1 What We Measure

Define features only after the objective and risk assumptions are explicit.

Chapter 3

Machine Learning

From Netflix recommendations to self-driving cars and the stock market, machine learning (ML) is quietly shaping the systems we use every day. But what is it, really?

At its heart, machine learning is about teaching computers to learn from experience. This is similar to teaching a child to recognize a cat by showing them lots of pictures and giving feedback. Instead of writing long, rigid lists of rules (like "if it has pointy ears and whiskers, it's a cat"), we let the computer discover the patterns in the data and improve its own performance over time.

Today, ML powers everything from voice assistants to medical diagnosis. In finance, it helps spot fraud, optimize portfolios, and predict future market direction. But financial machine learning has a unique challenge: unlike a clear picture of a cat, market data is incredibly noisy.

That is why the secret to algorithmic trading isn't just building a complex model; it's getting the data right to begin with. As you saw in the last chapter, high-quality, stationary data that preserves market memory is the real fuel for any financial ML system.

In this chapter, we'll focus on the most practical and widely used branch for our pipeline: supervised learning. We'll also briefly introduce unsupervised and reinforcement learning, so you see the big picture. You'll learn what makes each approach unique, how algorithms use "decision trees" to mimic human logic, and how they fit into the system you've already started building. By the end, you'll be ready to judge whether your model is actually uncovering a true market signal, or just memorizing historical flukes.

3.1 Principles of Machine Learning

Before we dive into the mechanics of machine learning, let us pause and consider how algorithms already shape our daily lives. Every time you unlock your phone with your face, get a movie suggestion from Netflix, or follow a recipe to bake cookies, you are relying on algorithms at work. These are not just abstract computer concepts; they are the invisible instructions guiding everything from your morning routine to the apps you use. In fact, we have all been using algorithms our entire lives, often without realizing it. Whether you are tying your shoes, solving a Rubik's cube, or deciding what to eat for breakfast, your daily routines are full of these stepwise procedures. So, what exactly is an **algorithm**? Think of it as a recipe: a clear, step-by-step set of instructions for turning raw ingredients (your input) into a finished dish (your output). In machine learning, algorithms are the backbone: they tell the computer exactly how to turn real-world data into useful predictions or decisions.

In traditional programming, a developer writes explicit rules that the computer must follow, a series of `if-then-else` statements and conditions that form the basis of a program. Want to detect spam email? A traditional programmer might write: "*if the subject line contains the word 'FREE' and the sender is unknown, mark as spam.*" This works reasonably well until spammers start writing "FR$$" instead of "FREE," and the programmer has to update the rules again. The fundamental limitation is clear: *the programmer must anticipate every case in advance.*

Machine learning (ML) flips this model on its head. Rather than a human writing the rules, we give the computer *examples* of correct behavior and let the computer *figure out the rules itself.* Feed it thousands of spam emails and thousands of legitimate ones, and the algorithm will discover, on its own, what patterns distinguish the two! This shift, from rule-writing to example-showing, is the core idea that makes machine learning so powerful and so different from everything that came before it (see Figure 3.1). Traditional programming requires exact instructions and requires human programmers to anticipate every possible scenario. In traditional programming, data and rules go *in*, and answers come *out*. In machine learning, data and answers (examples) go *in*, and *rules* come out. The output of the training process is not a prediction but a model—a compact mathematical representation of patterns in the data that can later be used to make predictions on new inputs.

Given how central machine learning has become, it's natural to wonder: if the field has existed since the 1950s, why are we only now seeing it become

so widespread? The practical answer lies in two major limitations that held the field back for decades: the availability of large, high-quality datasets, and the sheer amount of computing power required. At its core, machine learning compensates for a lack of explicit rules by leveraging large amounts of data. For much of its history, however, such data simply did not exist. The advent of cloud computing and advances in computer technology have enabled researchers and companies not only to store and process vast amounts of data, but also to access it seamlessly from anywhere in the world over the internet. Companies across the world had a collective awakening to the importance of data collection and information modeling. This shift was not just about better algorithms, but about the sudden availability of massive datasets and the computing power, especially GPUs, needed to process them [1, 3]. To put this explosion in perspective: in 2010, the world generated about 2 zettabytes of data over the entire year [4, 3]. Today, we generate roughly 402 million terabytes of data every single day [4]. For scale, one zettabyte is equal to one billion terabytes [2]. Data collection has become so desirable that companies are even offering hardware products for free, simply to gather consumer usage data. This relentless pursuit of information underscores how valuable data has become in the modern economy, fueling advances in machine learning and finance alike.

For decades, computing was all about the Central Processing Unit (CPU). The CPU was the "chef" in the kitchen, following recipes (programs) step by step, one dish at a time. Programs were written, compiled or interpreted, and executed via the CPU, which excelled at handling a wide variety of tasks, but only a few at once. Incremental improvements in CPU performance have mostly come from smarter designs—such as adding more cores (moving from single-core to quad-core and beyond), refining how instructions are processed, and squeezing out more parallelism where possible. But even with these advances, CPUs remain general-purpose processors. They are excellent at juggling many different tasks, but they are not designed for the massive parallel number-crunching that modern machine learning demands.

For the last twenty years, it was widely claimed by the media, politicians, and parents, that the video game industry wouldn't amount to much. Ironic, isn't it? The same industry, once dismissed as a frivolous pastime, became the unlikely hero in the story of machine learning's rise. Why? Because the relentless quest for ever more realistic graphics, such as lifelike shadows, explosions, and shimmering water, drove the development of the Graphical Processing Unit (GPU). Rendering a single video game frame requires calculating the color and brightness of millions of pixels simultaneously. Each calculation

is simple, but there are so many of them that only a processor designed for massive parallelism can keep up. This native design, centered around parallel processing, is perfectly suited for machine learning. Training a modern ML model rarely requires solving one giant, complicated equation. Instead, it involves doing millions of simple math operations, like matrix multiplications, at the same time. If a CPU is like one brilliant professor, a GPU is a stadium filled with 10,000 diligent students. Individually, each student can only do basic arithmetic, but working together, they can crunch through mountains of data.

When researchers realized they could repurpose gaming GPUs to train machine learning models, the field exploded. Algorithms that used to take months to train on a traditional computer could suddenly be completed in a few hours. Today, with the rise of cloud computing, this barrier has been significantly lowered. You no longer need to buy a supercomputer to run advanced financial models.

This democratization of cheap, powerful computation has opened the door for financial machine learning, allowing even small teams to build and test sophisticated trading algorithms that once required the resources of Wall Street giants.

3.1.1 What Makes a Machine Learn

One of the most puzzling aspects of machine learning is the word "learning" itself. When we say a machine "learns," we don't mean it in the same way a human does—there's no sudden flash of insight, no "aha!" moment. Instead, the process is more like tuning an old analog radio: you rotate the dial slowly, listening for the signal to grow clearer, making tiny adjustments until the static fades and the station locks in. Each small turn is guided by what you just heard—more static means turn back; cleaner signal means keep going.

But how do we even begin to describe what a machine is doing when it "learns"? This is where a bit of mathematical formalism becomes not just helpful, but essential. Imagine you want to teach a computer to predict tomorrow's stock price, or whether an email is spam. You could try to write down a giant list of rules, but the real world is far too messy for that—there are simply too many possibilities, too much noise, and too many exceptions. Instead, we turn to a more general, abstract way of thinking: we imagine there is some unknown function f that takes an input x (like today's market data, or the words in an email) and produces an output y (tomorrow's price, or a

spam/not-spam label). Mathematically, we write this as:

$$y = f(x)$$

This little equation is deceptively powerful. It says: whatever the task, there is some mapping from inputs to outputs, and our job is to discover (or approximate) the function f that best captures the patterns in the data. But here's the catch: we almost never know what f actually is! If we did, we wouldn't need machine learning at all—we'd just write down the formula and be done. The art and science of machine learning is all about searching for a good f—one that does well on the data we've seen, and hopefully generalizes to new data we haven't seen yet. This is why machine learning is fundamentally an optimization problem: we are searching for the function f (from a vast space of possible functions) that best fits our data.

But how does the machine "learn" this function? The process starts with a collection of background data—think of it as a set of practice problems. The model uses algorithms (its "study strategies") to search for patterns in this data. Each time it makes a prediction, it checks how close it was to the correct answer, much like a student checking their answers in the back of the textbook. Here's where the "learning" comes in: after each attempt, the model measures its error (how far off it was from the correct answer) and then adjusts its internal parameters to do better next time. This is a feedback loop—predict, check, adjust, repeat. Over many cycles, the model gradually improves.

This feedback-driven process is the heart of machine learning. The machine isn't developing intuition or understanding in the human sense; it's methodically tuning itself to minimize errors, one step at a time. In the end, what we call "learning" is really a process of continuous, data-driven improvement.

3.2 Supervised Learning

Historically, epigraphy (the study of ancient inscriptions) was a painstakingly manual process, requiring years of linguistic expertise and comparison with known texts like the Rosetta Stone. The process involved meticulous analysis of symbols, their contexts, and their relationships to one another. Researchers would document each symbol, cross-reference it with known languages, and gradually piece together the meanings. In 2017, Ubisoft, a video game company, launched a research project called *The Hieroglyphics Initiative* to coincide with the release of their game Assassin's Creed Origins (which is set in ancient Egypt).

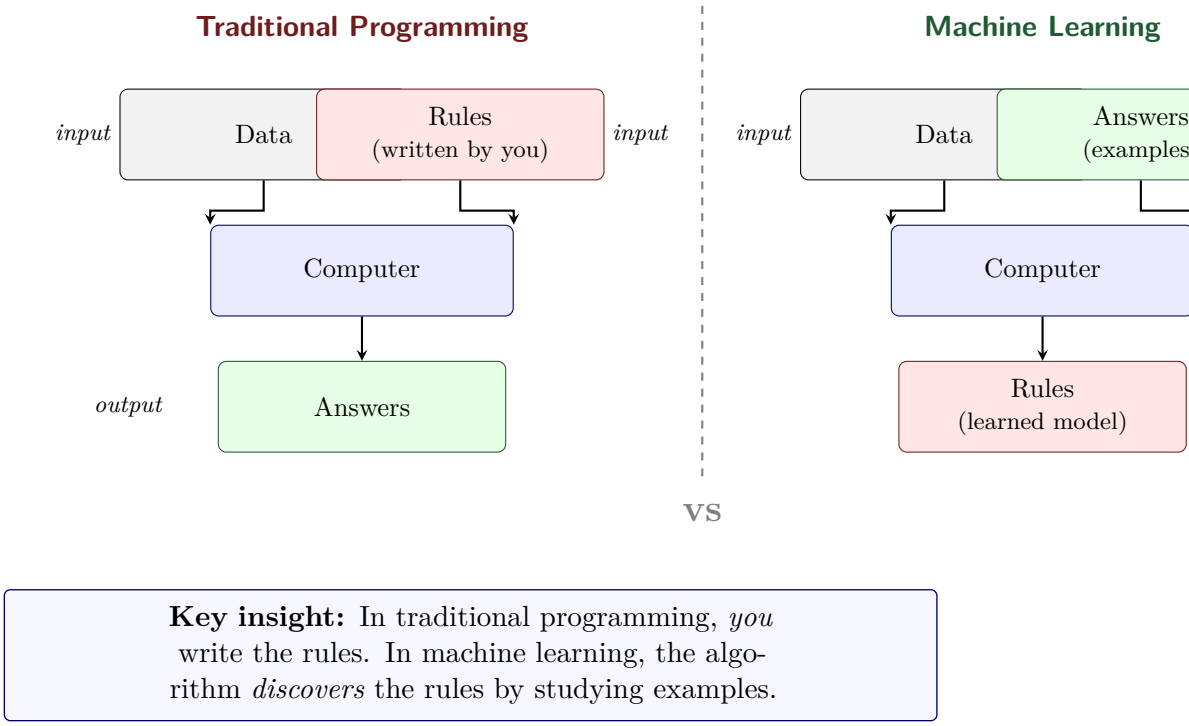


Figure 3.1: Traditional programming versus machine learning. On the left, the programmer supplies both data and hand-coded rules; the computer produces answers. On the right, the programmer supplies data and example answers; the computer produces the rules (a trained model).

Their goal was to see if machine learning could streamline the cataloging of the pharaohs' language. They did this by training a machine learning model on a large set of examples, like a stack of flashcards with both the hieroglyph and its translation. What took scholars centuries, the model accomplished in minutes—processing thousands of images, flagging uncertain symbols for human review, and transforming the pace of epigraphy. This is the promise of machine learning: not just speed, but the ability to scale human expertise beyond what was ever possible. The model performed remarkably well, accurately identifying the majority of hieroglyphs and demonstrating that AI could significantly accelerate the study and translation of ancient texts while still relying on human expertise for ambiguous cases.

This application of machine learning is known as **supervised learning**, where the model is trained on a labeled dataset (x), learning to map inputs (like images of hieroglyphs) to outputs (their translations) (y). This pattern, learning from examples with known answers, underpins everything from image recognition to predicting tomorrow's stock price.

Supervised learning is a type of machine learning where the model is trained

on a **labeled dataset**, meaning that each input data point is paired with the correct output (see Figure 3.1).

Table 3.1: A labeled dataset: each input x (hieroglyph image) is paired with its correct output y (translation). This is the raw material the supervised learning model trains on.

Input x **(Hieroglyph)**	**Label** y **(Translation)**
	Eye of Horus
	Owl (m)
	Water (n)
	Viper (f)
	Bread (t)
	Lion (r)
	Reed (i)
	Rope (h)

The model learns to map inputs to outputs by finding patterns in the data. Once trained, the model can make predictions on new, unseen data. This labeled dataset is broken into two key pairs: **features** and **labels**. You can see an example of this in Table 3.2.

Features are the measurable properties or characteristics of the data, while labels are the outcomes or targets that we want to predict. In the hieroglyph example, the features might include the number of curved strokes, straight strokes, whether the glyph has eyes, whether it represents an animal, and whether it is vertical. The label is the correct translation of the glyph.

This is a classic **classification** problem: the model learns to assign each glyph to a discrete category ("Owl," "Water," "Viper," etc.) based on its features. In other words, given the measurable properties of a glyph, the model predicts which class (translation) it belongs to. This mirrors the most common use of supervised learning in finance, where the goal is to classify market conditions

or trading opportunities into categories like “profitable” or “unprofitable.”

Table 3.2: Features and labels extracted from a labeled hieroglyph dataset. Rather than feeding the raw image directly to the model, we first describe each glyph in terms of measurable **features** — the properties the model can reason about. The **label** is the correct answer we want the model to learn to predict.

	Features x					**Labe**
Glyph	**Curved strokes**	**Straight strokes**	**Has eye**	**Animal**	**Vertical**	**Transl**
	5	2	Yes	No	No	Eye Horu
	3	4	Yes	Yes	No	Owl (
	6	0	No	No	No	Water
	4	3	No	Yes	No	Viper
	1	3	No	No	No	Bread
	2	5	Yes	Yes	No	Lion
	0	2	No	No	Yes	Reed
	2	6	No	No	No	Rope

Note: These features are simplified for illustration. In practice, a computer vision model would extract hundreds of such measurements automatically, but the principle is identical. Each row is one training example; the model’s job is to learn which combinations of feature values reliably predict each label.

Classification is the primary tool we’ll use in this course, because it allows us to make robust, actionable predictions about market or asset behavior without getting bogged down in the noise of trying to predict exact prices. Car manufacturers, for example, classify their vehicles into categories like “sedan,” “SUV,” or “truck” based on features like size, shape, and engine type. When we see a new car, we are usually able to generalize and classify it correctly, even if we haven’t seen that exact make and model before.

Regression is the other main branch of supervised learning. In regression, the model predicts a continuous value (such as the next price of an asset or tomorrow’s temperature) based on input features. Regression is used when we need to answer questions like “how much” or “to what extent.” While regression is powerful and widely used in many fields, in finance it is often

less practical due to the unpredictable nature of markets. That's why, in this book, we focus mainly on classification.

We know what we want the model to predict. The question now is how it learns to do it.

If you remember a game called "20 Questions," you can relate to how a model learns. In this game, one player thinks of an object, and the other player asks yes/no questions to narrow down the possibilities. In a similar way, a machine learning model learns by asking questions about the features in the data. The strategies a player might use in 20 Questions are like the different algorithms we can use to learn from a dataset, each with its own strengths and weaknesses:

- **Decision Trees:** Simple yet effective algorithms that split the data into branches based on feature values, making decisions at each node.
- **Random Forests:** An ensemble method that combines multiple decision trees to improve accuracy and reduce overfitting.
- **Gradient Boosting:** Another ensemble technique that builds trees sequentially, where each new tree corrects the errors of the previous ones.
- **Neural Networks (CNNs / Transformers):** Used for complex structured data such as images, language, or sequences.

In the next section, we'll dive into the differences of these learning techniques and how they are implemented in practice. We'll also discuss how to evaluate the performance of these models and ensure they generalize well to new data, rather than just memorizing historical patterns.

3.3 Decision Trees

Have you ever played 20 Questions with a friend? If so, you've already used a decision tree!

A decision tree is just a hierarchical structure (top-down) that splits the data into branches based on feature values, making decisions at each node. A model reaches a decision by traversing a decision tree from the **root node** to a **leaf node**, where the final prediction is made.

The root node is the topmost node in a decision tree, representing the entire dataset. A leaf node, on the other hand, is a terminal node that provides the final output or prediction for a given input. It's a node without any child nodes

(branches), and it represents the end of a decision path in the tree. Along the way, the model passes through internal nodes. Think of these as the decision points where you ask, "Does it live in water?" or "Is it man-made?". Internal nodes have one or more child nodes and represent the features used to split the data. In the example visualized in Figure 3.2, a 5-level decision tree is used to play the game of 20 Questions. The root node in this case is *Is it a living thing?*. There are several internal nodes such as *Does it live in water?*, *Is it an animal?*, and *Is it man-made?*. The leaf nodes represent the final answers, such as *whale*, *tree*, or *rock*.

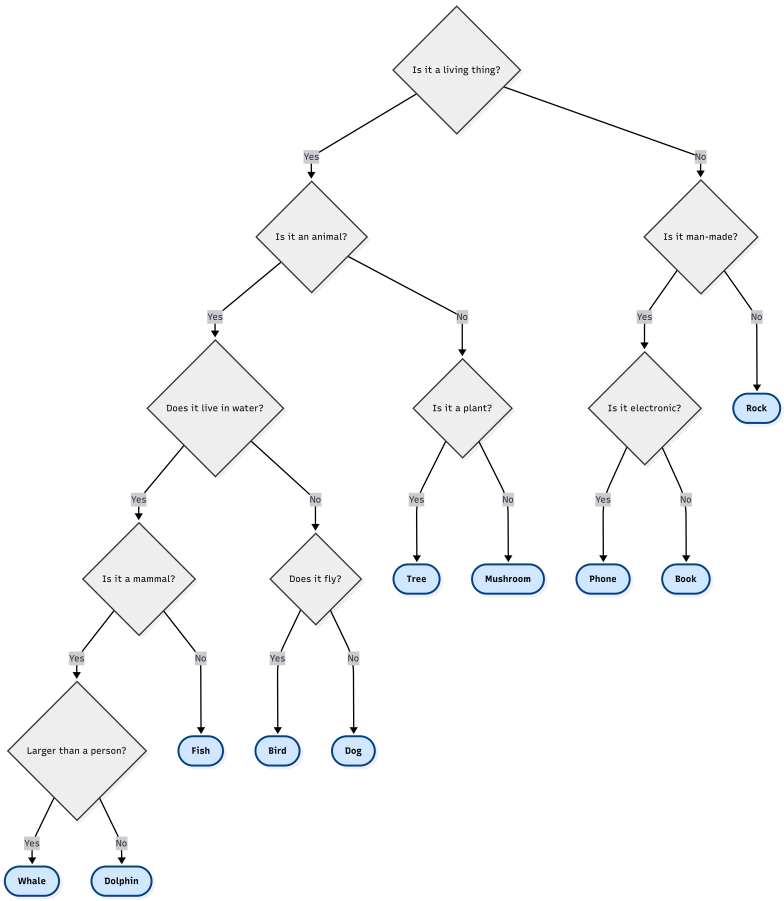
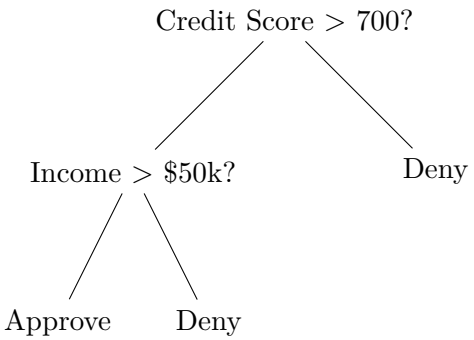


Figure 3.2: A 5-level decision tree for 20 Questions.

Decision trees are popular in financial machine learning because they allow for easy interpretation of the model's decision-making process. They also allow for sequential decision-making based on the features of the data, which is particularly useful in trading strategies where decisions are often made based on a series of conditions. This idea is central to one of the main benefits of decision trees: They are great at capturing complex, non-linear relationships between features, labels, and outcomes.

Let's consider an example of a bank that wants to predict whether a customer should be approved for a loan. The bank has historical data on previous loan applicants, including features like credit score, employment history, debt-to-income ratio, income, and the purpose of the loan. At the root node, the tree might start by asking: "Is the applicant's credit score above 700?" If the answer is yes, the tree follows one branch; if no, it follows another. At the next node, it might ask: "Is the applicant's annual income greater than $50,000?" Again, the answer determines the next branch. This process continues, with each internal node representing a question about a feature (like debt-to-income ratio or employment status), until the tree reaches a leaf node that gives the final decision: "Approve loan" or "Deny loan."



Of course, real-world decision trees can be much deeper and more complex, but the core idea remains the same: at each step, the tree asks a question about the data, and the answer determines the next step, until a final decision is reached.

Instead of writing a rigid rule like "always buy when the RSI is low," we can let a decision tree discover the nuanced relationship between momentum and volatility. The above example only accounts for two features, but in practice, a decision tree can handle dozens or even hundreds of features. This flexibility allows it to capture complex patterns in the data that might be missed by simpler, rule-based approaches.

3.3.1 Overfitting

Decision-tree learners are prone to a problem known as **overfitting**. In plain terms, overfitting happens when a model memorizes the training data, including all its random quirks and noise. Instead of learning the true underlying patterns, the model simply recalls the specifics of the training set. This is especially problematic in finance, where data is notoriously noisy and there are often thousands of potential features to consider. It's also arguable that many

of these features aren't even relevant to the trading strategy at hand. Let's generalize our example above. Suppose instead of just two features, we have a list of 10,000, and we want to predict whether a trade will be profitable. Now, our model is like a detective working on a cold case, sifting through mountains of evidence, searching for any clue that might crack the case. Sometimes, the detective finds a pattern that seems to explain everything, but it's really just a coincidence. And sometimes, the detective charges the wrong suspect. Worse, imagine that same detective becomes so convinced by the cold-case file that when a new witness walks in with fresh information, the detective dismisses it—the old file already has every answer. That is overfitting to a tee: the model has "solved" the training data so thoroughly that it can no longer learn from new evidence. By focusing too much on the training data and building an overly complex decision tree, the model loses sight of the bigger picture. It does this by creating branches for every little detail in the training data, even those that are irrelevant or random. The result is a hierarchical tree that becomes so many levels deep that it perfectly classifies the training data, but fails to generalize to new, unseen data. In finance, this can mean the difference between a model that looks like a genius in backtests and one that loses money in the real world. Overfitting was a major problem in the early days of financial machine learning, with quantitative researchers often reporting incredible trading accuracy in testing, only to be disappointed by a model being unable to translate the lessons learned to live trading.

3.3.2 Pruning

The general rule of thumb is a decision tree should be as simple as possible. In machine learning, one can abstract Sir Tony Hoare's famous quote[1]: there are two ways of constructing a machine learning model: one way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies. Overfitting is a classic example of the second approach. To combat this, we use a technique called **pruning**, which is like trimming a tree to remove unnecessary branches. Pruning is a process that removes branches that have little importance. Removing these branches helps strip away the noise. This is typically done using two techniques: **pre-pruning** and **post-pruning**. Pre-pruning is about halting the growth of the tree before it becomes too complex. Pre-pruning techniques include the following:

- **Maximum Depth:** Limit the depth of the tree to prevent it from

[1]Adapted from C. A. R. Hoare's 1980 Turing Award lecture on software complexity.

growing too complex.

- **Minimum Samples per Leaf:** Require a minimum number of data points in a leaf node before allowing a split.
- **Minimum Samples per Split:** Require a minimum number of data points in a node before allowing a split.
- **Maximum Features:** Limit the number of features considered for splitting at each node.

Post-pruning, on the other hand, allows the tree to grow fully and then trims it back afterward. Post-pruning techniques include the following:

- **Cost Complexity Pruning:** Uses a pruning parameter (α) to balance accuracy and size.
- **Reduced Error Pruning:** Removes branches that do not improve the overall accuracy of the model.
- **Minimum Impurity Decrease:** Only keeps branches that actually help the tree make better decisions; tiny improvements are ignored to keep things simple.
- **Minimum Leaf Size:** Makes sure each final decision in the tree is based on enough examples, so the model doesn't make rules from just a few odd cases.

In practice, pre-pruning is faster and post-pruning tends to produce slightly better trees—but both share the same goal: a model that generalizes rather than memorizes.

3.3.3 Choosing the Best Attribute

A single decision tree, carefully built and pruned, is a powerful tool — but as we've seen, it remains fragile. The next section puts that tree to work on real financial data, and in doing so, reveals exactly where its limits lie.

Now that we've seen how to keep our trees from growing wild, let's turn to the next big question: when the model reaches a decision point, how does it choose which feature to split on? Humans might be able to make this choice based on visual indicators, intuition, or experience, but a machine needs a more systematic approach. The ability to choose the best attribute is ultimately

the heart of what makes a decision tree smart, and it's where the math gets interesting.

The way a decision tree chooses the best attribute is by measuring how well each feature separates the data into distinct classes. To do this, it needs a way to judge how "pure" each group is after a split.

A node is considered pure based on how mixed or unmixed the data is in a group after a split:

- A node is "pure" if all the samples in it belong to the same class (for example, all are "cats" or all are "dogs").
- A node is "impure" if it contains a mix of different classes (some "cats" and some "dogs" together).

A decision tree tries to split the data so that each group (node) is as pure as possible—meaning each group contains mostly just one kind of thing. The purer the node, the easier it is for the tree to make a confident decision.

To measure this, decision trees use two main metrics: **Gini Impurity** and **Information Gain**. Both metrics quantify the "purity" of the nodes after a split, but they do so in slightly different ways.

Information Gain

Information gain relies on the concept of entropy, which measures the amount of uncertainty or disorder in a dataset. Think of entropy as a way to measure how "mixed up" or uncertain a group of data is. A high entropy means the data is very mixed (lots of different classes), while low entropy means the data is more ordered (mostly one class). It is scored on a simple scale from 0 to 1.

- **Perfect Purity** (Entropy = 0): The data is perfectly sorted. For example, a basket containing only apples has zero entropy because there is no uncertainty about what you will pull out.
- **Maximum Disorder** (Entropy = 1): The data is completely mixed. A basket containing exactly half apples and half oranges has an entropy of 1 because it is entirely unpredictable.

A decision tree's job is to take a messy, mixed-up dataset and organize it into clean, pure groups. To figure out the best way to divide the data, the tree relies on information gain.

Here is how the process works:

- **Measure Before:** The tree calculates the entropy (mixed-up-ness) of the data before making a split.
- **Measure After:** It then calculates the entropy after splitting the data into new groups.

The process of *information gain* is the difference in entropy before and after a split based on a given feature. The attribute that results in the largest reduction in entropy(closer to 0 being better in this case) is chosen as the best attribute to split on. Table 3.3 illustrates this process with our trusted hieroglyph example. The model evaluates each feature, calculates the information gain for each possible split, and selects the one that maximizes the gain. We can see that the feature "is an animal" provides the highest information gain, making it the best choice for the first split in our decision tree. From there, the process continues for each subtree, recursively applying the same logic until the tree is fully grown or pruned.

Gini Impurity

Imagine reaching into a bag of hieroglyph flashcards and trying to guess which glyph you will pull out. If the bag contains only Owl cards, your guess is easy. If it contains every glyph in equal proportion, your guess is essentially random, the bag is maximally impure. **Gini Impurity** formalizes exactly this intuition: it measures the probability that a randomly chosen sample from a node would be incorrectly classified if it were labeled at random according to the class distribution in that node.

Similar to entropy, Gini Impurity ranges from 0 to 1, where 0 indicates perfect purity and 1 indicates maximum impurity. In plain English, Gini impurity is just a way for the model to ask: "How likely am I to be wrong if I guess the label of something in this group at random?" If every card in the bag is the same, you can't be wrong—impurity is zero. If the bag is a total jumble, your chances of guessing wrong go up, and so does the impurity. The lower the Gini impurity, the more confident the model can be in its decisions. In practice, the model uses this number to help decide which questions (splits) will do the best job of sorting things into neat, predictable groups.

Entropy vs Information Gain

Both entropy and Gini impurity serve the same purpose: they measure how mixed or pure a node is after a split. They both share the ultimate goal of

Table 3.3: Evaluating candidate splits on the hieroglyph dataset (see Table 3.2). For each feature, the glyphs are divided into two groups based on the split condition. A good split produces groups where each contains mostly one class — the tree prefers the feature that results in the purest groups, quantified by **Information Gain**.

Feature	**Split**	**Group A** (*Yes*)	**Group B** (*No*)	**Purity**
Animal?	Animal = Yes	Owl Viper Lion	Eye Water Bread Reed Rope	**High**
Has Eye?	Has Eye = Yes	Eye Owl Lion	Water Viper Bread Reed Rope	**Medium**
Vertical?	Vertical = Yes	Reed	Eye Owl Water Viper Bread Lion Rope	**Medium**
Curved ≥ 3	Strokes ≥ 3	Eye Owl Water Viper Lion	Bread Reed Rope	**Low**

Note: "Purity" here is qualitative — in the next section we will replace this judgment with a precise number using **Gini Impurity** and **Information Gain**. The *Animal?* feature produces the cleanest separation: Group A contains exactly the three animal glyphs, and Group B contains everything else. A decision tree formalizes this intuition by selecting the feature with the highest information gain — the split that reduces entropy the most.

guiding the model toward splits that create cleaner, more distinct groups.

So, which one should you use? In practice, both metrics often lead to similar results, with the choice between them coming down to the specific dataset and the preferences of the practitioner.

3.4 Building the Financial ML Pipeline

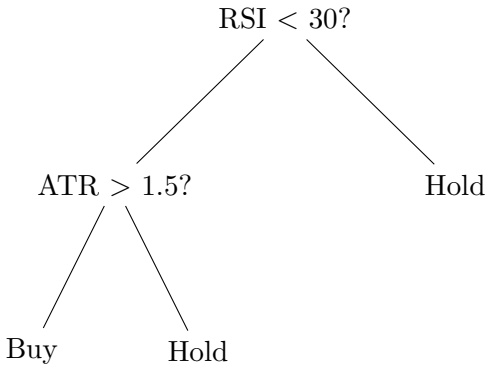
Up to this point, we've built a model that can ask questions about features, but we haven't yet said what those features are or where they come from. That's the real challenge this section resolves: connecting the abstract mechanics of decision trees to the concrete, engineered features that actually drive financial predictions.

Table 3.4: scenarios where one metric may be more practical

Scenario	**Gini Impurity**	**Entropy (Information Gain)**
Split Behavior	Favors dominant classes; tends to isolate the largest group first	Produces more balanced partitions across classes
Dataset Size	Efficient for large, high-dimensional datasets	Better suited to structured datasets with balanced classes
Sensitivity	Less sensitive to small probability differences	sensitive to subtle shifts in class distribution

Rule of thumb: When in doubt, start with Gini, it is faster and the default in most production libraries. Switch to entropy if your classes are well-balanced and you want the tree to weight minority classes more carefully.

In prior chapters, we focused on transforming raw market data into dollar volume bars, then applied fractional differentiation to enforce stationarity while preserving memory. Our next step is to convert this processed data into a **feature matrix**, which serves as the input for our decision tree model. A feature matrix is a structured representation of our data. You have actually already seen a feature matrix in Table 3.2, where each row corresponds to a different glyph and each column corresponds to a different feature. In the context of financial machine learning, each row would represent a different time point or market event, and each column would represent a different market indicator or feature derived from our processed data. This is the process of translating our market hypotheses into the structured inputs our decision tree will use to learn. The jump from hieroglyph translation to algorithmic trading is smaller than it might seem. Just as an epigraphist maps glyphs to their translations, a quantitative model evaluates market indicators to assess the probability of a profitable trade. To make this concrete, consider a simple trading strategy built on just two features: the Relative Strength Index (RSI), which measures momentum, and the Average True Range (ATR), which measures volatility. Instead of writing a rigid rule like "always buy when the RSI is low," we can let a decision tree discover the nuanced relationship between momentum and volatility. Here is what such a tree might look like:



This tiny tree shows how a model can combine two simple indicators to make a trading decision. In practice, a decision tree can handle dozens or even hundreds of features, capturing complex patterns in the data that might be missed by simpler, rule-based approaches.

The most important practical question for any trading model is: what are we actually trying to predict? While it might seem natural to use a **regression** model to forecast the exact future price of an asset, in practice this is rarely effective. Financial markets are notoriously unpredictable, and the extreme difficulty of accurately predicting precise prices makes regression a poor fit for most trading strategies.

Instead, we turn to **classification**. But classification only works if we can define clear, actionable labels for our data. The key design decision is to label each example by whether a trade would have been profitable or not. This reframes the problem: rather than asking "how much will the price move?" we ask "was this trade a good idea?" The classification in this case is a binary label, either the trade was profitable (1) or it was not (0). This approach has several advantages:

- Simplifies the prediction task: the model focuses on learning patterns that lead to successful trades, not on forecasting exact prices.
- Aligns with the real goal of trading: making profitable decisions, not just predicting numbers.
- Provides clear, interpretable outputs: traders can easily understand predictions in terms of real-world profit or loss.
- Makes evaluation straightforward: profitability is determined simply by comparing entry and exit prices in historical data.

To simplify this binary classification problem, we can define a **target variable** that indicates whether a trade was profitable or not. The question we ask the

model is: "Given the market conditions at this bar, will the next bar close higher (profitable) or lower (unprofitable)?" This target variable is the artifact that we append to our feature matrix, and for simplicity, we can define it as follows:

Target variable definition:

- **1** = profitable trade (next bar closes higher than the current bar)
- **0** = unprofitable trade (next bar closes equal or lower than the current bar)

With our end goal of building a feature matrix for classification in mind, we can now outline the data pipeline that transforms raw market data into a structured format suitable for machine learning. Before we build out our feature matrix, we need to dive into the calculations that we can use to generate the features themselves, known as **technical indicators**.

3.4.1 Technical Indicators

Technical indicators are mathematical calculations based on historical price, volume, or open interest data that traders use to predict future market behavior. We have seen some of these indicators in action already, such as the Relative Strength Index (RSI), Simple Moving Average (SMA), and Exponential Moving Average (EMA). These indicators are designed to capture different aspects of market behavior, such as momentum, volatility, and trend direction. Generally, technical indicators can be categorized into five main types:

- **Trend/momentum:** These indicators help identify both the direction and speed of the market.
- **Mean Reversion:** These indicators help identify when the market is likely to revert to its mean or average price level.
- **Volatility:** These indicators measure the rate of price change and the degree of variation in trading prices over time.
- **Statistical Moments:** These indicators try to identify the shape and spread of the price distribution, such as skewness and kurtosis.
- **Volume/Liquidity:** These indicators incorporate trading volume to measure the cumulative flow of money into and out of an asset, helping to confirm the strength of a price trend.

Financial data is extremely noisy; the market is often cited as being 90% noise and 10% signal. Indicators act as mathematical filters to suppress high-frequency noise and hopefully reveal low-frequency trends. A single indicator is rarely enough to capture the full complexity of the market. The true power of technical indicators in modern algorithmic trading comes from the ability to combine multiple indicators into a single feature matrix, allowing the model to learn from a richer set of signals.

3.4.2 From Raw Data to Features

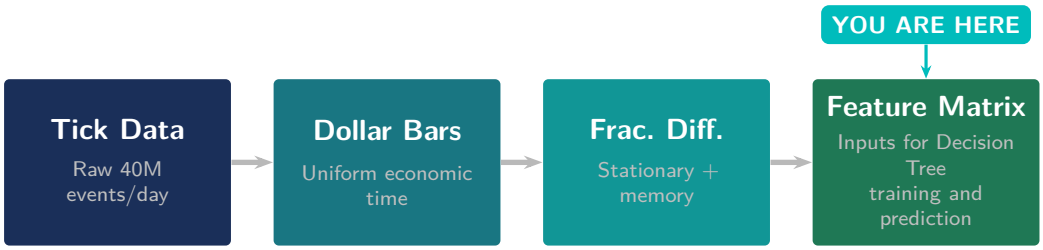


Figure 3.3: The course data pipeline. Each stage transforms the raw market data into a form the machine learning model can use. *Tick Data* captures every trade event; *Dollar Bars* resample that stream into uniform economic time; *Fractional Differentiation* enforces stationarity while preserving trend memory; and the *Feature Matrix* encodes our trading hypotheses as measurable inputs. This chapter picks up at the final stage.

If we recall the data pipeline from Chapter **??**, we can see how each stage transforms the raw market data into a form that is closer to what our machine learning model can use. Building a feature matrix is the final step in this pipeline, where we take the processed data and encode our trading hypotheses as measurable inputs for the model. Our feature matrix is essentially a spreadsheet where every row is one dollar volume bar and every column is one measurement about that moment in time. Our algorithm will read across each row and ask "given all these signals at this moment, did the market go up or down next?"

Our feature matrix might look something like this (see Table 3.5):

In this case, we add the timestamp and the close price of the bar to provide the wider context. In the next step, when we move to training, we use the close price as a comparison point to determine whether the trade was profitable or not, which will be encoded in the label column. The datetime column is useful for tracking the temporal sequence of the data, which can be important for time series analysis and ensuring that our model respects the chronological order of events.

Table 3.5: A financial feature matrix: each row represents one dollar volume bar, and each column represents a measurable signal at that moment in time. The columns are **features** x

	Features x			
Timestamp	**Close**	**EMA_14**	**RSI_14**	**ATR_14**
09:32:01	148.20	147.80	54.2	1.32
09:34:47	149.50	148.10	61.7	1.41
09:38:03	148.90	148.30	58.4	1.38
		⋮		

Note: In practice the feature matrix will contain hundreds of rows (one per dollar bar) and potentially dozens of columns.

With our current feature matrix, we have one half of the supervised learning equation: the features x.

$$y = f(x)$$

The other half is the labels y, which we will define based on whether the trade following each bar was profitable or not.

3.4.3 Finding Labels: Fixed-Horizon Method

To calculate labels, the most intuitive approach is the fixed-horizon method. Here, we simply compare the close price of the current bar with the close price a fixed number of bars into the future. For example, for a 5-bar horizon, we compare the close price now with the close price 5 bars later. If the future close price is higher, we label that row as a profitable trade (1); otherwise, it is unprofitable (0).

This method assumes we are looking for trades that will be profitable in a short-term window, which is a common approach in algorithmic trading. In standard machine learning literature, it is common to create the y label by looking exactly h bars into the future (e.g., 5 bars) and evaluating if the price went up or down. The fixed-horizon method is simple and easy to implement, making it a popular baseline.

3.4.4 Finding Labels: The Triple-Barrier Method

Modern financial machine learning often uses a more sophisticated approach, known as the *Triple-Barrier Method* (López de Prado). The Triple-Barrier Method sets three thresholds (or barriers) for the price movement:

- **Upper Barrier:** A profit-taking limit (1).
- **Lower Barrier:** A stop-loss limit (0).
- **Time Barrier:** A maximum holding period (which acts as a timeout).

Unlike the simple fixed-horizon method, the Triple-Barrier Method allows for more nuanced labeling of trades. A fixed-time horizon ignores the fluctuations and the path the asset took to reach the future price. If a stock crashes and recovers within the 5-bar window, a fixed-horizon method might label it as profitable, even though the trade would have been risky and potentially unprofitable in practice. Imagine a scenario where a trader enters a position, and the price immediately drops by a staggering 50%. In our idealistic fixed-horizon, this temporary drop is effectively ignored; in practice, it's likely a broker might liquidate the trader's position far before the 5-bar window is up, resulting in a loss.

In comparison, the Triple-Barrier Method takes into account the path the price takes, and labels the trade based on which barrier is hit first. The algorithm reads forward bar-by-bar and assigns the y label based on which of these three barriers is touched first. This method is particularly useful in financial markets, as y is not necessarily determined by looking at a single future timestamp. Instead, y is determined by tracking the asset's price path forward from the current bar until it hits a defined profit target, a stop-loss limit, or a maximum time expiration. The maximum expiration time is customizable, with most practitioners using a specific number of days (e.g., 1 day), or a larger number of observations (e.g., 100 bars), to ensure that the model is not overly sensitive to short-term fluctuations.

3.4.5 Meta Labeling

In our current architecture, a single decision tree asked to predict whether a trade will be profitable is carrying two jobs at once: deciding the *direction* of the trade (long or short), and deciding whether the trade is worth taking at all. These are not the same question, and forcing one model to answer both tends to produce a model that is mediocre at each.

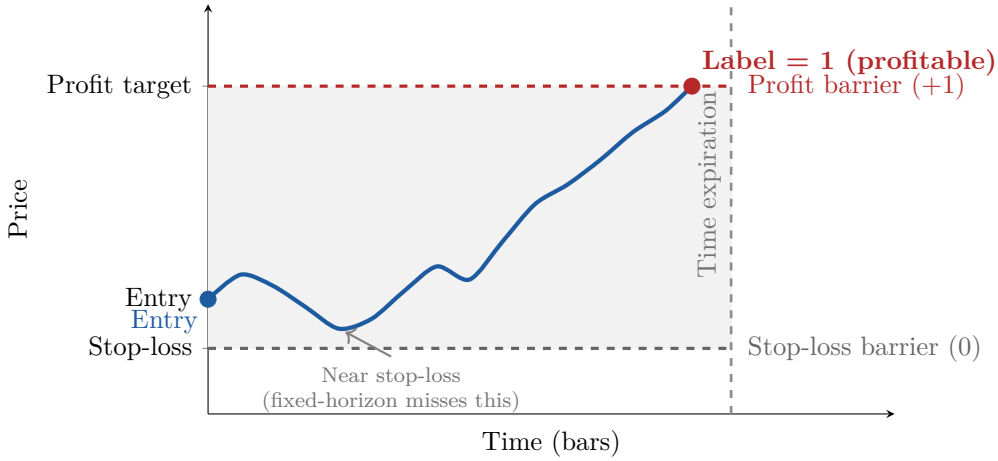


Figure 3.4: The Triple-Barrier Method. A trade opens at the entry point (blue dot) and the price path is tracked bar-by-bar within a corridor bounded by a profit barrier (red, dashed), a stop-loss barrier (gray, dashed), and a time expiration (vertical, dashed). The label y is assigned based on whichever barrier the price hits first. Here, the price dips dangerously close to the stop-loss before recovering and eventually hitting the profit barrier—earning label $y = 1$. A fixed-horizon method would assign the same label without ever "seeing" the near-miss, illustrating why the path matters for realistic trade labeling.

Think of how a trading desk is actually organized. A senior trader reads the chart, studies the indicators, and calls the direction: "I think this stock is going up." That is a judgment about the market. Before a single dollar is committed, a risk manager reviews the same setup and asks a different question entirely: "Given our current exposure and the quality of this signal, should we actually take this trade?" The trader does not decide position size; the risk manager does not call market direction. The two roles are deliberately separated because the skills required are different.

Meta-labeling formalizes exactly this division of labor. Rather than asking one model to be right about everything, we build a two-stage architecture:

- **Primary Model (the trader):** A rule-based signal — a moving average crossover, an RSI threshold, or any technical indicator — that determines the *side* of the trade: Long (+1) or Short (−1). This model is simple and fast. It does not need to be perfect; it just needs to identify a direction worth investigating.

- **Secondary Model, the Meta-Label (the risk manager):** A decision tree trained not to predict market direction, but purely to predict whether the primary model's signal will be correct. Its output is binary:

take the bet (1) or pass (0). This is the model that decides whether to act on the primary signal at all.

The power in this setup is that we train the secondary model on the *performance* of the primary model, rather than on the market itself. This abstraction allows us to essentially ask: "Given what the primary model just said, how much should I trust it?"

To make this concrete, suppose our primary model is a simple RSI crossover rule: buy when RSI crosses above 30 (oversold recovery), short when RSI crosses below 70 (overbought reversal). We run this rule over our historical dollar bars and record every trade signal. For each signal, we then check, using the Triple-Barrier labels from the previous section, whether the primary model was actually right. Those outcomes become our meta-labels:

- **Meta-label = 1:** The primary model called the correct direction *and* the Triple-Barrier outcome was a win.
- **Meta-label = 0:** The primary model was wrong, or the Triple-Barrier outcome was a loss or timeout.

We then train our decision tree on these meta-labels, using the same feature matrix of technical indicators as inputs. This two-stage architecture allows us to strip away more of the noise in the market. We train our model to predict the *quality* of the primary signal, rather than the market itself. This allows our decision tree to recognize the conditions under which the RSI signal is likely to succeed, and more importantly, when it is likely to fail. Figure 3.5 shows how these two stages connect.

3.4.6 Finalizing the Feature Matrix

The structure of our feature matrix is now complete: each row represents a dollar bar, each column represents a technical indicator, and the final column contains the label indicating whether the trade was profitable or not.

We can see an example of this structure in Table 3.6.

Yet, we are missing the important step of actually calculating our feature matrix and labels across the entire dataset. We know the *blueprint* of our model, but we need to build our training and testing datasets that will allow us to both train and evaluate our model's performance. Our goal is to loop over our dataframe, calculate the technical indicators for each bar, and then apply the triple-barrier method to assign the labels.

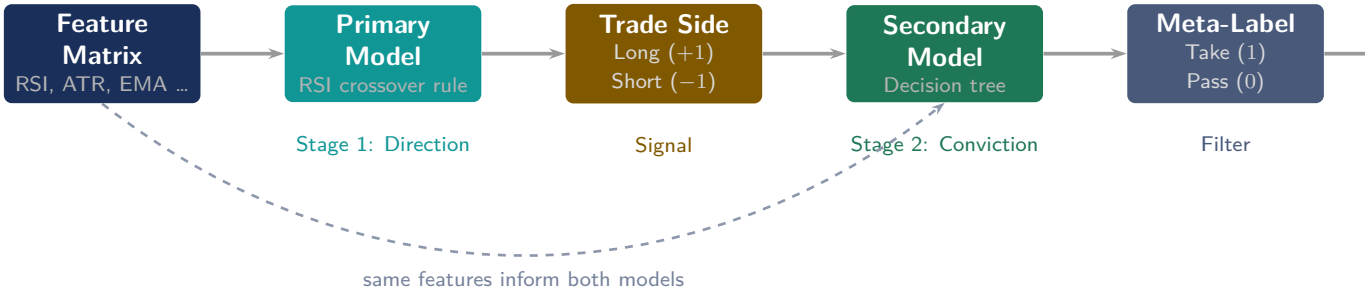


Figure 3.5: The Meta-Labeling architecture. The **Primary Model** (Stage 1) uses a rule-based signal to determine the *direction* of the trade (long or short). The **Secondary Model** (Stage 2) then acts as a filter, predicting whether the primary signal is likely to be correct. The dashed arc highlights that both models draw from the same feature matrix, but answer fundamentally different questions.

A Python implementation of this process can be seen in Listing 3.6. The feature matrix is built in a single pass through the dataframe, with each feature being calculated and appended to the row dictionary. The calculations for these are abstracted away via appropriate functions. [2] The final step in the loop is to apply the triple-barrier method to assign the label for that bar, which is then appended to the row dictionary.

It's important to understand that two boundaries shape the loop. The lower bound `lookback` ensures every bar has enough price history behind it for the indicators to be valid. This helps add weight to the model's predictions, and makes the features we are using, such as the 14-period EMA (which relies on this past history), more meaningful. The upper bound `len(bars) - horizon` ensures the triple-barrier method always has enough bars ahead to resolve. Without this upper bound, the Triple-Barrier window would run off the end of the dataset—the price path would have no future bars to evaluate, and the label could never be assigned.

Every iteration of the loop produces a single row of the feature matrix, which we use to build the whole dataset. Once the loop completes, we have finally assembled our complete feature matrix. The next question is how to divide this dataset so the model is tested honestly—and here, financial data requires a different approach than most textbooks describe.

[2] Popular Python libraries for technical indicators include `ta-lib`, `pandas-ta`, and `finta`.

Table 3.6: The complete feature matrix: features x on the left describe market conditions at each dollar bar; the label y on the right records whether the trade that followed was profitable.

Features x					**Label y**
Timestamp	**Close**	**EMA_14**	**RSI_14**	**ATR_14**	**Profitable?**
09:32:01	148.20	147.80	54.2	1.32	**1**
09:34:47	149.50	148.10	61.7	1.41	**0**
09:38:03	148.90	148.30	58.4	1.38	**1**
09:41:22	147.60	148.20	48.3	1.45	**0**
09:45:10	150.10	148.50	65.1	1.29	**1**
		⋮			

Label definition: $y = 1$ if the close price of the next bar is higher than the current bar (profitable); $y = 0$ otherwise (unprofitable).

3.5 Evaluating the Model

A single decision tree, carefully built and pruned, is a powerful tool — but as we've seen, it remains fragile. The next section puts that tree to work on real financial data, and in doing so, reveals exactly where its limits lie.

We now understand the core steps required to transition our raw market data into a structured feature matrix. Finally, after several data engineering steps, we are ready to train our decision tree model. Althouth laborious, the importance of this preperation process cannot be overstated. Without properly engineered features, even the most sophisticated machine learning model will struggle to make accurate predictions. Companies have spent millions of dollars on data engineering teams to ensure that their models are fed with high-quality, relevant features. Our next step is to start the actual training process. This section dives into the practical aspects of training a decision tree model, including how to split our dataset into training and testing sets, how to evaluate the model's performance, and how to interpret the results in a financial context.

3.5.1 Train/Test Split

Before we train our model, we need to understand why splitting a dataset into training and testing sets is essential. In machine learning, our goal is to

```python
feature_matrix = []
for i in range(lookback, len(bars) - horizon):
    # -- Features x: market conditions at bar i --
    row = {
        'timestamp' : bars[i].timestamp,
        'close'     : bars[i].close,
        'ema_14'    : compute_ema(bars, i, period=14),
        'rsi_14'    : compute_rsi(bars, i, period=14),
        'atr_14'    : compute_atr(bars, i, period=14),
    }
    # -- Label y: apply Triple-Barrier Method --
    label = triple_barrier(
        bars     = bars,
        start    = i,
        upper    = bars[i].close * (1 + profit_target),
        lower    = bars[i].close * (1 - stop_loss),
        max_bars = horizon,
    )
    row['label'] = label  # 1 = profit barrier hit first
                          # 0 = stop-loss or time barrier hit first
    feature_matrix.append(row)
```

Figure 3.6: Building the feature matrix row by row. For each dollar bar `i`, we compute the technical indicators that describe market conditions at that moment, then apply the Triple-Barrier Method to assign the label y.

assess how well the rules a model learned from training data generalize to new, unseen data. We cannot travel forward in time to test on tomorrow's market, so we do the next best thing: simulate the future by holding back a portion of today's data.

We split our dataset into two parts. The **training set** is used to build the model; the **testing set** is withheld entirely during training and used only to evaluate performance afterward. The testing set acts as a proxy for future data, giving us an honest estimate of how the model will perform in live trading.

3.5.2 The Confusion Matrix

With our model trained, we can now evaluate its performance on the testing set. The standard tool for this is a **confusion matrix**: a table that compares the model's predicted labels against the true labels, revealing where it succeeds and—more importantly—where it fails.

We break the model's predictions into four categories:

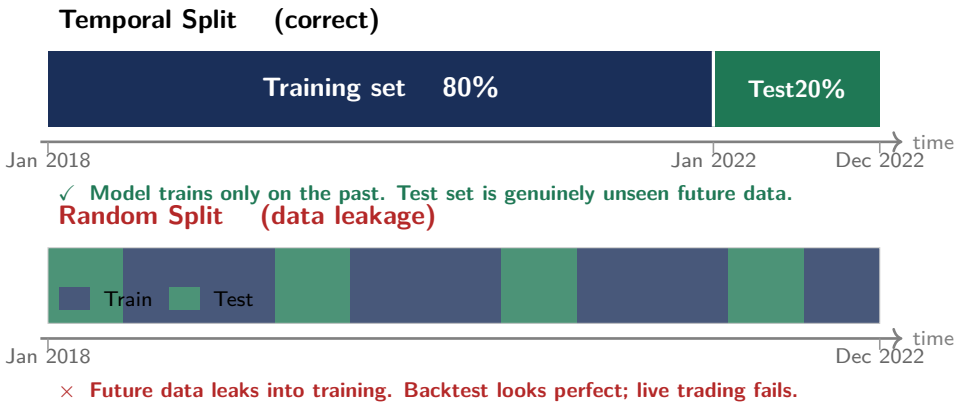


Figure 3.7: Two approaches to splitting the feature matrix into training and test sets. **Top (correct):** A temporal split preserves chronological order — the model trains on 2018–2021 data and is evaluated on 2022 data it has never seen. **Bottom (wrong):** A random shuffle interleaves future and past rows, allowing the model to learn from data that postdates the observations it is supposedly predicting. This is **data leakage**: the backtest appears excellent, but the model has seen the future and will fail immediately in live trading.

- **True Positives (TP):** The model predicted a profitable trade, and it was indeed profitable—a gain of capital.
- **False Positives (FP):** The model predicted a profitable trade, but it was actually unprofitable—a loss of capital.
- **True Negatives (TN):** The model predicted an unprofitable trade, and it was indeed unprofitable—capital preserved.
- **False Negatives (FN):** The model predicted an unprofitable trade, but it was actually profitable—a missed opportunity.

This four-way breakdown is crucial in finance, where the cost of being wrong is not symmetric. Strip away stocks, bonds, and assets, and investing is essentially a form of sophisticated wagering: you stake capital on the future performance of some asset. A loss doesn't just erase a gain—it erodes your base.

Consider a trade with 1.4-to-1 odds. If you risk \$100 and win, you collect \$140—not bad. But if you lose, the entire \$100 is gone. The reward for being right is \$40 of profit; the penalty for being wrong is \$100 of loss. This **asymmetric risk** is why the categories that matter most in our confusion matrix are the False Positives and False Negatives: these are the trades that cost real money.

Identifying True Positives and True Negatives matters, but they are not as

financially consequential. Let us return to our hieroglyph example. Suppose the training set was not in fact curated by hundreds of experts, but by a single student using MSPaint. With a compromised dataset, it is entirely plausible that the model would misclassify a large number of glyphs. An example confusion matrix for this scenario is shown in Figure **??**.

3.5.3 Precision, Recall, and the Cost of Being Wrong

The confusion matrix gives us the raw counts, but counts alone don't tell us whether the model is *useful*. A model that never predicts a profitable trade has zero False Positives but also zero True Positives—it would never lose money, but it would never make any either. To move from counts to actionable insight, we need three derived metrics: **precision**, **recall**, and the **F1-score**.

Precision answers the question: *of all the trades the model said were profitable, how many actually were?*

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

A model with high precision rarely cries wolf—when it signals "buy," you can trust it. Low precision means the model generates many false alarms, each of which costs real capital.

Recall answers the complementary question: *of all the trades that were actually profitable, how many did the model catch?*

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

A model with high recall misses few opportunities—it casts a wide net. Low recall means the model is overly cautious, leaving money on the table by passing on trades that would have been winners.

In finance, precision and recall pull in opposite directions. A trader who bets on every signal will have high recall but low precision (many bets, many losses). A trader who only bets on the strongest signals will have high precision but low recall (few bets, few losses, but also few wins). The **F1-score** balances the two:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1-score is the harmonic mean of precision and recall. It is high only when *both* metrics are high, making it a single number that penalizes a model for being lopsided in either direction.

Which metric matters most depends on your trading strategy. If capital preser-

vation is paramount (e.g., a risk-averse fund), optimize for precision: accept fewer trades but make sure they count. If capturing every opportunity matters more (e.g., a high-frequency strategy that can absorb small losses), optimize for recall. In most cases, the F1-score provides a reasonable starting point for comparing models, but the final choice should always reflect the financial objectives and risk tolerance of the strategy.

Principles and Pitfalls

- **Why classification over regression?** Predicting exact prices is extraordinarily difficult in noisy markets. Classification simplifies the task to a yes/no decision,*is this trade worth taking?*,which aligns directly with how trading profits are realized.

- **What if our indicators are correlated?** If two features carry nearly the same information (e.g., RSI and stochastic oscillator), the tree gains little from the second split. Highly correlated features can inflate the apparent importance of a signal without improving predictions. Feature selection and dimensionality reduction help.

- **What if our labels are imbalanced?** If 90% of bars are labeled unprofitable and only 10% profitable, a model can achieve 90% accuracy by always predicting "unprofitable", and never make a single trade. Metrics like precision, recall, and F1-score exist precisely to catch this trap.

Bibliography

[1] IBM. Bringing big data to the enterprise, 2012. Accessed: 2026-03-10.

[2] National Institute of Standards and Technology. Prefixes for binary multiples, 2024. Accessed: 2026-03-10.

[3] David Reinsel, John Gantz, and John Rydning. Worldwide global datasphere forecast, 2021–2025. Technical report, IDC, 2021. Accessed: 2026-03-10.

[4] Statista. Volume of data/information created, captured, copied, and consumed worldwide from 2010 to 2025, 2024. Accessed: 2026-03-10.